can also use the `nameof()` operator to further reduce the chance of typos, such as by calling `GetSection(nameof(MapSettings))`.

Each call to `Configure<T>` sets up the following series of actions internally:

1. Creates an instance of `ConfigureOptions<T>`, which indicates that `IOptions<T>` should be configured based on configuration.

   If `Configure<T>` is called multiple times, multiple `ConfigureOptions<T>` objects will be used, all of which can be applied to create the final object in much the same way that `IConfiguration` is built from multiple layers.

2. Each `ConfigureOptions<T>` instance binds a section of `IConfiguration` to an instance of the `T` POCO class, setting any public properties on the options class based on the keys in the provided `ConfigurationSection`.

   Remember that the section name (`"MapSettings"` in listing 10.8) can have any value; it doesn't have to match the name of your options class.

3. The `IOptions<T>` interface is registered in the DI container as a singleton, with the final bound POCO object in the `Value` property.

This last step lets you inject your options classes into handlers and services by injecting `IOptions<T>`, as you saw in listing 10.7, giving you encapsulated, strongly typed access to your configuration values. No more magic strings. Woo-hoo!

WARNING If you forget to call `Configure<T>` and inject `IOptions<T>` into your services, you won't see any errors, but the `T` options